Tower Defense – Arbeitspakete

Ihr entwickelt gemeinsam ein **Tower Defense Spiel** in WPF (C# / .NET 8).
Der Basiscode (Spielschleife, Rendering, Pfad, Grid) ist bereits fertig.
Eure Aufgabe: die Spiellogik implementieren – verteilt auf Arbeitspaare.

Spielprinzip

- Gegner laufen entlang eines vorgegebenen Pfades von links nach rechts.
- Ihr platziert Türme neben dem Pfad, die automatisch auf Gegner schießen.
- Erreicht ein Gegner das Ende, verliert ihr ein Leben.
- Tötet ihr einen Gegner, erhaltet ihr Punkte.
- Das Spiel endet wenn alle Leben aufgebraucht sind.

Git-Workflow (für alle Teams verbindlich)

```
1.  git pull                              ← immer zuerst aktuellen Stand holen
2.  git checkout -b feature/wp-XX-kurzbeschreibung
3.  Implementierung + regelmäßige Commits
4.  git push -u origin feature/wp-XX-kurzbeschreibung
5.  Pull Request auf GitHub öffnen → main als Ziel-Branch
6.  Ein anderes Team reviewed den PR (mind. 1 Approval)
7.  Merge in main
```

Branch-Namensschema: `feature/wp-01-enemy-move`, `feature/wp-05-tower-shoot`, ...

Commit-Nachrichten: kurz und aussagekräftig auf Deutsch oder Englisch.

```
☑  "WP1: Enemy.Move() implementiert"
☑  "Wegpunkt-Logik korrigiert – Index-Überlauf behoben"
✗  "fix"  |  ✗  "asdf"  |  ✗  "ich hab was gemacht"
```

Abhängigkeiten zwischen Paketen

```
WP1 (Move) ──► WP8/WP9 (Spawner) braucht lebende Gegner
WP2 (Damage) ──► WP7 (Kollision) ruft TakeDamage auf
WP3 + WP4 ──► WP5 (Shoot) baut darauf auf
WP5 ──► WP6 + WP7 (Projektil existiert erst dann)
WP10 ──► WP18/WP19 (Turm-Subklassen müssen existieren)
WP11 ──► alle anderen (Bereinigung setzt funktionierende Logik voraus)
```

Koordiniert euch: Teams mit abhängigen Paketen sollten früh kommunizieren.

Minimalset – ohne diese 12 Pakete läuft das Spiel nicht

Die folgende Reihenfolge ist empfohlen, da jedes Paket auf dem vorherigen aufbaut:

Schritt	Paket	Was wird damit möglich?
1	**WP13** UI-Labels	Score/Leben/Welle werden angezeigt
2	**WP02** TakeDamage	Gegner können Schaden nehmen
3	**WP08** StartNextWave	Welle kann gestartet werden
4	**WP09** WaveSpawner.Update	Gegner werden gespawnt
5	**WP01** Enemy.Move	Gegner bewegen sich auf dem Pfad
6	**WP04** CanShoot	Türme haben einen Schuss-Cooldown
7	**WP03** FindTarget	Türme finden Gegner in Reichweite
8	**WP05** Tower.Shoot	Türme feuern Projektile ab
9	**WP06** Projectile.Move	Projektile fliegen über den Canvas
10	**WP07** CheckCollision	Projektile treffen Gegner
11	**WP10** TryPlaceTower	Türme können platziert werden
12	**WP11** GameState.Update	Tote Gegner/Projektile werden entfernt, Score zählt

Alle anderen Pakete (WP12–WP21) sind **optionales Polish** und können unabhängig danach implementiert werden.

Die 19 Arbeitspakete

WP01 – Gegner-Bewegung `Enemy.Move()`

Datei: `Models/Enemy.cs`
Branch: `feature/wp-01-enemy-move`

Implementiere die Methode `Move()` in der Klasse `Enemy`.

Der Gegner soll sich entlang der Wegpunkte in `PathDefinition.Waypoints` bewegen:

1. Berechne den Vektor vom aktuellen `Position` zum nächsten Wegpunkt (`Waypoints[_waypointIndex]`).
2. Berechne die Distanz (Pythagoras).
3. Ist die Distanz kleiner als `Speed`: springe zum nächsten Wegpunkt (`_waypointIndex++`).
   - Wenn kein nächster Wegpunkt mehr existiert: `ReachedEnd = true`, `IsAlive = false`.
4. Normalisiere den Vektor und multipliziere mit `Speed`.
5. Aktualisiere `Position`.

Testbar wenn: Gegner bewegen sich nach Klick auf „Nächste Welle" sichtbar auf dem Canvas.

WP02 – Schaden nehmen `Enemy.TakeDamage()`

Datei: `Models/Enemy.cs`
Branch: `feature/wp-02-enemy-damage`

Implementiere `TakeDamage(int damage)`:

1. Ziehe `damage` von `Hp` ab.
2. Stelle sicher dass `Hp` nicht unter 0 fällt (`Math.Max`).
3. Wenn `Hp == 0`: setze `IsAlive = false`.

Testbar wenn: Der HP-Balken über Gegnern schrumpft wenn Projektile treffen (erfordert WP5–WP7).

WP03 – Ziel suchen `Tower.FindTarget()`

Datei: `Models/Tower.cs`
Branch: `feature/wp-03-tower-find-target`

Implementiere `FindTarget(List<Enemy> enemies)`:

1. Iteriere über alle Gegner mit `IsAlive == true`.
2. Berechne Distanz zwischen `Position` (Turm) und `enemy.Position`.
3. Merke den Gegner mit der geringsten Distanz, **der innerhalb von `Range` liegt**.
4. Gib diesen Gegner zurück, oder `null` wenn keiner in Reichweite ist.

Testbar wenn: Türme drehen sich (intern) in Richtung Gegner – sichtbar erst nach WP5.

WP04 – Schuss-Cooldown `Tower.CanShoot()`

Datei: `Models/Tower.cs`
Branch: `feature/wp-04-tower-cooldown`

Implementiere `CanShoot(double deltaTime)`:

1. Wenn `_cooldown > 0`: reduziere `_cooldown -= deltaTime`.
2. Gib `true` zurück wenn `_cooldown <= 0`.

Die Methode `Shoot()` setzt den Cooldown zurück: `_cooldown = 1.0 / FireRate`.
(Das passiert in WP5 – koordiniert euch.)

Testbar wenn: Türme schießen in gleichmäßigen Abständen (nach WP5).

WP05 – Schießen `Tower.Shoot()`

Datei: `Models/Tower.cs`
Branch: `feature/wp-05-tower-shoot`

Implementiere `Shoot(Enemy target)`:

1. Setze Cooldown zurück: `_cooldown = 1.0 / FireRate`.
2. Berechne Richtungsvektor von `Position` zu `target.Position`.
3. Normalisiere den Vektor (teile durch Länge).
4. Erstelle `new Projectile(Position, new Vector(ndx, ndy), speed: 300, Damage)`.
5. Gib das Projektil zurück.

Abhängigkeit: Benötigt WP4 (CanShoot). Projektil erscheint erst wenn WP6 implementiert ist.

WP06 – Projektil-Bewegung `Projectile.Move()`

Datei: `Models/Projectile.cs`
Branch: `feature/wp-06-projectile-move`

Implementiere `Move(double deltaTime)`:

1. Berechne neue Position:
   `Position = new Point(Position.X + Direction.X * Speed * deltaTime, ...)`

2. Setze `IsActive = false` wenn das Projektil den Canvas verlässt:
   `x < -50 || x > 850 || y < -50 || y > 530`

Testbar wenn: Gelbe Kugeln fliegen sichtbar über den Canvas (nach WP5).

WP07 – Kollisionserkennung `Projectile.CheckCollision()`

Datei: `Models/Projectile.cs`
Branch: `feature/wp-07-projectile-collision`

Implementiere `CheckCollision(List<Enemy> enemies)`:

1. Für jeden lebenden Gegner: berechne Distanz zwischen `Position` und `enemy.Position`.
2. Wenn Distanz < 18 (Treffradius):
   - `enemy.TakeDamage(Damage)` aufrufen.
   - `IsActive = false` setzen.
   - `return` – ein Projektil trifft nur einmal.

Abhängigkeit: Benötigt WP2 (TakeDamage).

WP08 – Welle starten `WaveSpawner.StartNextWave()`

Datei: `Models/WaveSpawner.cs`
Branch: `feature/wp-08-wave-start`

Implementiere `StartNextWave()`:

1. `CurrentWave++`
2. `_enemiesToSpawn = 5 + CurrentWave * 2`
3. `_spawnInterval = Math.Max(0.4, 1.2 - CurrentWave * 0.05)` (schneller mit jeder Welle)
4. `IsSpawning = true`, `_spawnTimer = 0`

Testbar wenn: „Nächste Welle"-Button löst tatsächlich eine Welle aus (nach WP9).

WP09 – Gegner spawnen `WaveSpawner.Update()`

Datei: `Models/WaveSpawner.cs`
Branch: `feature/wp-09-wave-spawner`

Implementiere `Update(double deltaTime)`:

1. Wenn `!IsSpawning || _enemiesToSpawn <= 0`: return `null`.
2. `_spawnTimer -= deltaTime`. Wenn `_spawnTimer > 0`: return `null`.
3. `_spawnTimer = _spawnInterval`.
4. `_enemiesToSpawn--`. Wenn 0: `IsSpawning = false`.
5. Spawne Gegner je nach Welle:
   - `CurrentWave % 3 == 0` → `new TankEnemy()`
   - `CurrentWave % 2 == 0` → `new FastEnemy()`
   - Sonst → `new Enemy(hp: 80 + CurrentWave * 15, speed: 1.2, reward: 10)`
6. Gib den neuen Gegner zurück.

Abhängigkeit: Benötigt WP8. TankEnemy / FastEnemy benötigen WP16 / WP17.

WP10 – Turm platzieren `GameState.TryPlaceTower()`

Datei: `Models/GameState.cs`
Branch: `feature/wp-10-place-tower`

Implementiere `TryPlaceTower(Point mousePosition, TowerType selectedType)`:

1. Berechne Zelle: `col = (int)(mousePosition.X / CellSize)`, analog `row`.
2. Validierung (return false wenn):
   - Außerhalb des Grids
   - `GridHelper.IsCellOnPath(col, row)` == true
   - `_occupiedCells.Contains((col, row))` == true
3. Berechne Pixel-Mittelpunkt der Zelle.
4. Erstelle Turm je nach `selectedType` (Basic / Sniper / MachineGun).
5. Füge zu `Towers` und `_occupiedCells` hinzu. return `true`.

Abhängigkeit: SniperTower (WP18) und MachineGunTower (WP19) müssen existieren.

WP11 – Spielzustand bereinigen `GameState.Update()`

Datei: `Models/GameState.cs`
Branch: `feature/wp-11-gamestate-update`

Implementiere `Update(Canvas canvas)`:

1. Iteriere über `Enemies.ToList()`:
   - Wenn `!enemy.IsAlive`: `RemoveFromCanvas`, aus Liste entfernen.
   - Wenn `ReachedEnd`: `Lives--`, sonst `Score += enemy.Reward`.
2. Iteriere über `Projectiles.ToList()`:
   - Wenn `!p.IsActive`: `RemoveFromCanvas`, aus Liste entfernen.
3. Wenn `Lives <= 0`: `IsGameOver = true`.

Wichtig: Immer `ToList()` verwenden, damit die Liste während der Iteration modifiziert werden darf.

WP12 – Reichweiten-Visualisierung (MainWindow)

Datei: `MainWindow.xaml.cs` → Methode `GameCanvas_MouseMove`
Branch: `feature/wp-12-range-circle`

Implementiere das Hover-Feedback für Türme:

1. Mausposition bestimmen: `e.GetPosition(GameCanvas)`.
2. Suche Tower in `_state.Towers` mit Distanz < 20 zur Mausposition.
3. Falls gefunden:
   - Alten `_rangeCircle` aus Canvas entfernen.
   - Neuen `Ellipse` erstellen: `Width = Height = tower.Range * 2`.
   - `Fill`: `new SolidColorBrush(Color.FromArgb(40, 255, 255, 255))`.
   - `Stroke`: `Brushes.White`, `StrokeThickness = 1`.
   - Positionieren und zum Canvas hinzufügen.
4. Falls kein Turm: `_rangeCircle` entfernen, auf `null` setzen.

WP13 – UI-Labels aktualisieren (MainWindow)

Datei: `MainWindow.xaml.cs` → Methode `UpdateUI()`
Branch: `feature/wp-13-ui-labels`

Implementiere `UpdateUI()`:

```
TxtLives.Text = $"♥  Leben:  {_state.Lives}";
TxtScore.Text = $"★  Score:  {_state.Score}";
TxtWave.Text  = $"〜  Welle:  {_state.WaveSpawner.CurrentWave}";
BtnNextWave.IsEnabled = !_state.WaveSpawner.IsSpawning && !_state.IsGameOver;
```

Einfach aber wichtig – ohne diese Labels gibt es keine Rückmeldung über den Spielstand.

WP14 – Game-Over-Overlay (MainWindow)

Datei: `MainWindow.xaml.cs` → Methode `ShowGameOver()`
Branch: `feature/wp-14-game-over`

Implementiere das Game-Over-Overlay:

1. Halbtransparentes schwarzes Rechteck (800×480) über den Canvas legen.
2. Großer „GAME OVER"-TextBlock (FontSize ≥ 48, Farbe Rot), zentriert.
3. Zweiten TextBlock mit erreichtem Score darunter.
4. `_timer.Stop()` aufrufen (bereits vorgegeben – nicht löschen).

Positionierung mit `Canvas.SetLeft` / `Canvas.SetTop`.

WP15 – Tower-Info-Panel (MainWindow / XAML)

Dateien: `MainWindow.xaml` + `MainWindow.xaml.cs`
Branch: `feature/wp-15-tower-info`

Erweitere die Seitenleiste um ein dynamisches Info-Panel:

1. Füge in `MainWindow.xaml` drei `TextBlock`-Elemente unter den RadioButtons hinzu: z. B. `TxtTowerRange`, `TxtTowerDamage`, `TxtTowerFireRate`.
2. Handle das `Checked`-Event jedes RadioButtons im Code-behind.
3. Aktualisiere die TextBlocks je nach gewähltem Turmtyp mit dessen Werten.

Beispiel Anzeige wenn „Sniper" gewählt:

```
Reichweite:   250 px
Schaden:       15
Schussrate:   0.5 / s
```

WP16 – FastEnemy

Datei: `Models/FastEnemy.cs`
Branch: `feature/wp-16-fast-enemy`

Vervollständige die Klasse `FastEnemy` (erbt von `Enemy`):

1. Überschreibe `GetColor()` → wähle eine passende Farbe (z. B. Orange).
2. Überschreibe `GetSize()` → kleinere Darstellung (≤ 20px).
3. **Zusatz:** Überschreibe `Move()` um eine leichte Schlangenlinien-Bewegung einzubauen (Sinus-Offset senkrecht zur aktuellen Bewegungsrichtung).

Der Konstruktor mit `base(hp: 40, speed: 3.5, reward: 5)` ist bereits fertig.

WP17 – TankEnemy

Datei: `Models/TankEnemy.cs`
Branch: `feature/wp-17-tank-enemy`

Vervollständige die Klasse `TankEnemy` (erbt von `Enemy`):

1. Überschreibe `GetColor()` → auffällige, dunkle Farbe (z. B. DarkViolet).
2. Überschreibe `GetSize()` → größere Darstellung (≥ 32px).
3. **Zusatz:** Überschreibe `Draw()` um einen dicken Außenrand zu zeichnen
   (Panzer-Optik: extra Ellipse mit großem Stroke).
   Tipp: `base.Draw(canvas)` zuerst aufrufen, dann darüber zeichnen.

Der Konstruktor mit `base(hp: 350, speed: 0.7, reward: 25)` ist bereits fertig.

WP18 – SniperTower

Datei: `Models/SniperTower.cs`
Branch: `feature/wp-18-sniper-tower`

Vervollständige die Klasse `SniperTower` (erbt von `Tower`):

1. Überschreibe `GetColor()` → z. B. `Brushes.DarkGreen`.
2. **Zusatz:** Überschreibe `FindTarget()` so dass der Sniper immer den Gegner anvisiert,
   der dem **Ziel am nächsten** ist (am weitesten auf dem Pfad fortgeschritten).
   Tipp: Du benötigst eine öffentliche Property `WaypointIndex` in `Enemy` – ergänze sie.

Der Konstruktor mit `base(position, range: 250, damage: 15, fireRate: 0.5)` ist fertig.

WP19 – MachineGunTower

Datei: `Models/MachineGunTower.cs`
Branch: `feature/wp-19-machinegun-tower`

Vervollständige die Klasse `MachineGunTower` (erbt von `Tower`):

1. Überschreibe `GetColor()` → z. B. `Brushes.OrangeRed`.
2. **Zusatz:** Nach 8 Schüssen soll eine **Überhitzungs-Pause** von 1.5 Sekunden eingelegt werden.
   - Privates Feld `_shotCount` zählt abgefeuerte Schüsse.
   - Überschreibe `Shoot()`: Schuss abfeuern, `_shotCount++`.
   - Wenn `_shotCount >= 8`: `_shotCount = 0`, forciere Cooldown auf 1.5 Sekunden.

Der Konstruktor mit `base(position, range: 100, damage: 40, fireRate: 4.0)` ist fertig.

WP20 – Spielstand speichern und laden

Dateien: `Services/SaveGameService.cs`, `Models/GameState.cs`, `MainWindow.xaml.cs`
Branch: `feature/wp-20-save-load`

Implementiere das Speichern und Laden des Spielstands als JSON-Datei (`savegame.json`).

WP20a – Speichern (`SaveGameService.Save`):

1. Erstelle ein `SaveGame`-Objekt mit `Lives`, `Score`, `CurrentWave` aus dem `GameState`.
2. Befülle `save.Towers` – für jeden Tower in `state.Towers` eine `TowerSaveData`-Instanz.
3. Ergänze dazu in `Tower` die Property `public TowerType TowerType { get; }` (im Konstruktor setzen).
4. Serialisiere mit `JsonSerializer.Serialize(save, JsonOptions)` und schreibe in Datei.
5. Log: `Log.Information("Spielstand gespeichert: ...")`.
6. Fehlerbehandlung: `try/catch` mit `Log.Error(...)`.

WP20b – Laden (`SaveGameService.Load`):

1. Prüfe ob Datei existiert (`File.Exists`), sonst `Log.Warning` und `return null`.
2. Lese JSON, deserialisiere zu `SaveGame`.
3. Log: `Log.Information("Spielstand geladen: ...")`.

WP20c – In MainWindow einbinden (`BtnSave_Click` / `BtnLoad_Click`):

1. `BtnSave_Click`: `SaveGameService.Save(_state)` aufrufen + MessageBox.
2. `BtnLoad_Click`:
   - `SaveGameService.Load()` aufrufen.
   - Spielfeld zurücksetzen.
   - `_state.LoadFrom(save)` aufrufen.
   - Türme aus `save.Towers` neu platzieren.
   - Außerdem: `WaveSpawner.SetWave(save.CurrentWave)` (neue Methode ergänzen).

Verwendete Klassen/Namespaces: `System.Text.Json`, `System.IO`, `TowerDefenseGame.Models`

WP21 – Logging mit Serilog

Dateien: `App.xaml.cs`, `Models/GameState.cs`, `Services/SaveGameService.cs`, `MainWindow.xaml.cs`
Branch: `feature/wp-21-logging`

Integriere strukturiertes Logging mit Serilog in das gesamte Spiel.
Serilog ist bereits als NuGet-Paket im Projekt eingebunden.

WP21a – Logger konfigurieren (`App.xaml.cs → OnStartup`):

```
Log.Logger = new LoggerConfiguration()
    .MinimumLevel.Debug()
    .WriteTo.Console()
    .WriteTo.File("towerdefense.log",
                  rollingInterval: RollingInterval.Day,
                  retainedFileCountLimit: 7)
    .CreateLogger();

Log.Information("=== Tower Defense gestartet ===");
```

WP21b – Log-Aufrufe in MainWindow (alle mit `// TODO (WP21b)` markiert):

Stelle	Level	Nachricht
Konstruktor	Information	„Hauptfenster geladen – Spiel bereit."
BtnNextWave	Information	„Welle {Wave} gestartet."

Stelle	Level	Nachricht
BtnRestart	Information	„Spiel neu gestartet."

WP21c – Log-Aufrufe in GameState (`Update()`):

Stelle	Level	Nachricht
Gegner besiegt	Debug	„Gegner besiegt – Reward {R}. Score: {S}."
Gegner am Ziel	Warning	„Gegner am Ziel. Leben: {L}"
Game Over	Warning	„GAME OVER – Score: {S}, Welle: {W}"
Turm platziert	Information	„Turm {Type} auf Zelle ({C},{R}) platziert."

WP21d – Log-Aufrufe in SaveGameService (bereits als Kommentar im Code vorhanden):
Aktiviere die auskommentierten `Log.xxx(...)`-Aufrufe.

Wichtig: `Log.CloseAndFlush()` in `App.OnExit()` ist bereits implementiert – nicht entfernen.

Übersichtstabelle

#	Paket	Datei	Schwierigkeit
WP01	`Enemy.Move()`	`Models/Enemy.cs`	★★★
WP02	`Enemy.TakeDamage()`	`Models/Enemy.cs`	★
WP03	`Tower.FindTarget()`	`Models/Tower.cs`	★★
WP04	`Tower.CanShoot()`	`Models/Tower.cs`	★★
WP05	`Tower.Shoot()`	`Models/Tower.cs`	★★★
WP06	`Projectile.Move()`	`Models/Projectile.cs`	★★
WP07	`Projectile.CheckCollision()`	`Models/Projectile.cs`	★★
WP08	`WaveSpawner.StartNextWave()`	`Models/WaveSpawner.cs`	★
WP09	`WaveSpawner.Update()`	`Models/WaveSpawner.cs`	★★
WP10	`GameState.TryPlaceTower()`	`Models/GameState.cs`	★★★
WP11	`GameState.Update()`	`Models/GameState.cs`	★★
WP12	Range-Visualisierung	`MainWindow.xaml.cs`	★★
WP13	UI-Labels	`MainWindow.xaml.cs`	★
WP14	Game-Over-Overlay	`MainWindow.xaml.cs`	★★
WP15	Tower-Info-Panel	`MainWindow.xaml(.cs)`	★★
WP16	`FastEnemy`	`Models/FastEnemy.cs`	★
WP17	`TankEnemy`	`Models/TankEnemy.cs`	★★
WP18	`SniperTower`	`Models/SniperTower.cs`	★★
WP19	`MachineGunTower`	`Models/MachineGunTower.cs`	★★★
WP20	Spielstand speichern/laden	`Services/SaveGameService.cs`	★★★
WP21	Logging (Serilog)	`App.xaml.cs`, `GameState.cs`, ...	★★